4

Data Transformation

April, 2026

Rachid BENOUINI

What is data transformation?

The Challenge

- Data collected from multiple sources often varies in format, structure, and scale, making analysis challenging
- Need for data transformation, the process of modifying data to fit a particular purpose or context

The Definition

- In other words, converts data from one format or structure into another
- Involves tasks such as cleaning, standardization, or aggregation, etc.

The Outcome

- Transformed data is structured and refined, making it more suitable for analysis, storage, or processing
- Enhances data quality, consistency, and usability

Real-World Impact

- Better results for decision-making in areas like business intelligence
- Better data for training in ML models

Data Transformation Categories

Aesthetic

"Improves the presentation of data without changing its semantic meaning"

- Standardize abbreviations (e.g., 'St' to 'Street')
- Convert texts to consistent cases (uppercase/lowercase)
- Format phone numbers, zip codes, etc., consistently
- Convert all dates to ISO format (YYYY-MM-DD)

Constructive

"Adds information to enhance data completeness and analytical value"

- Add new attributes from existing ones (e.g. total sales)
- Geocode addresses to add coordinates
- Merge data from multiple sources
- Impute missing values using statistical methods

Structural

"Reorganizes data structure for better efficiency and usability"

- Rename fields for clarity
- Reorder columns for logical flow
- Convert strings to numerics
- Create binary indicators from text
- Change data orientation (pivot/unpivot)

Destructive

"Removes unwanted or irrelevant data to improve data quality"

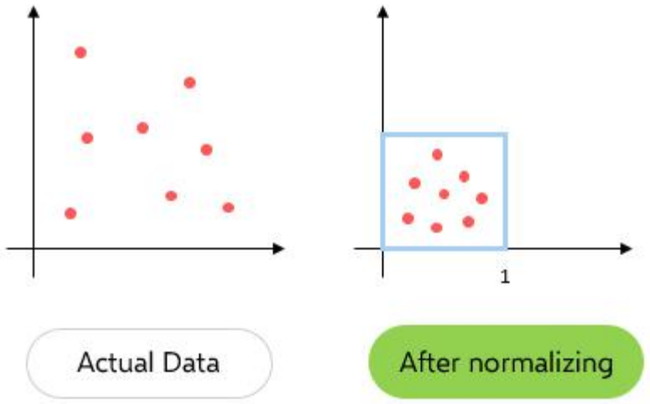
- Remove duplicates
- Drop records with excessive missing values
- Remove outliers
- Capping/winsorizing (replacing with threshold values)

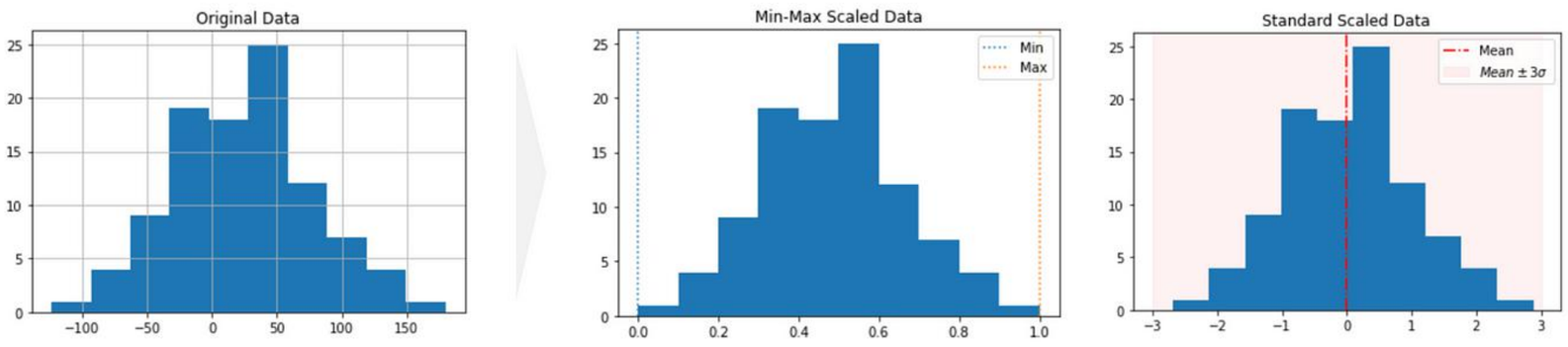
Prerequisites

Aspect	Definition
Scaling	• The process of adjusting the range of features in a dataset so that they are on a similar scale • Ensures features are comparable and properly weighted, especially when features have different units or scales of measurement
Features	• The input variables or attributes
Distribution	• The way values of a feature are spread or arranged • Shows how often different values or ranges of values occur in a dataset

Data Transformation Techniques: Normalization

- Normalization is the process of rescaling data to fit within a specific range, typically (0,1) or (-1,1).
- It ensures that all features have a **common scale**, preventing certain features from dominating due to larger numerical values.
- It is particularly useful when features in a dataset have varying scales, improving comparability and model performance.
- **Feature scaling** through normalization helps machine learning models converge faster and perform better.
- **Normalization does not change the distribution of the data but ensures values remain within a defined range.**

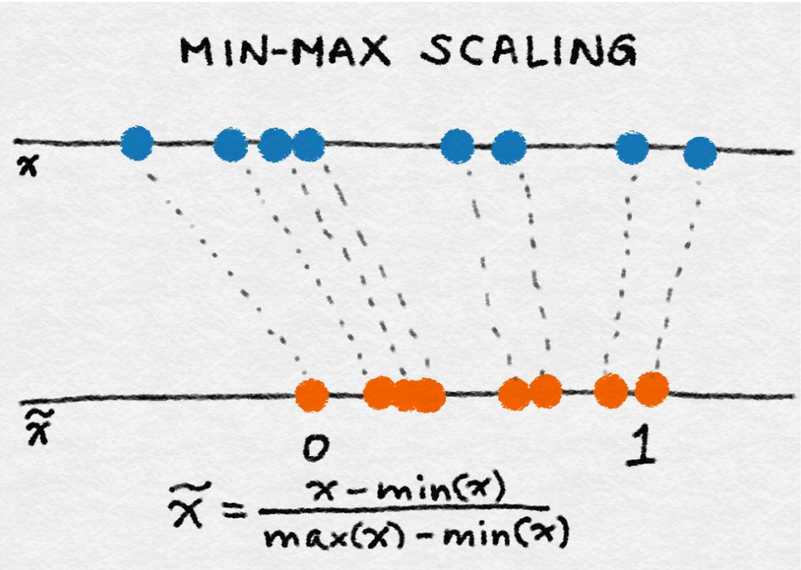




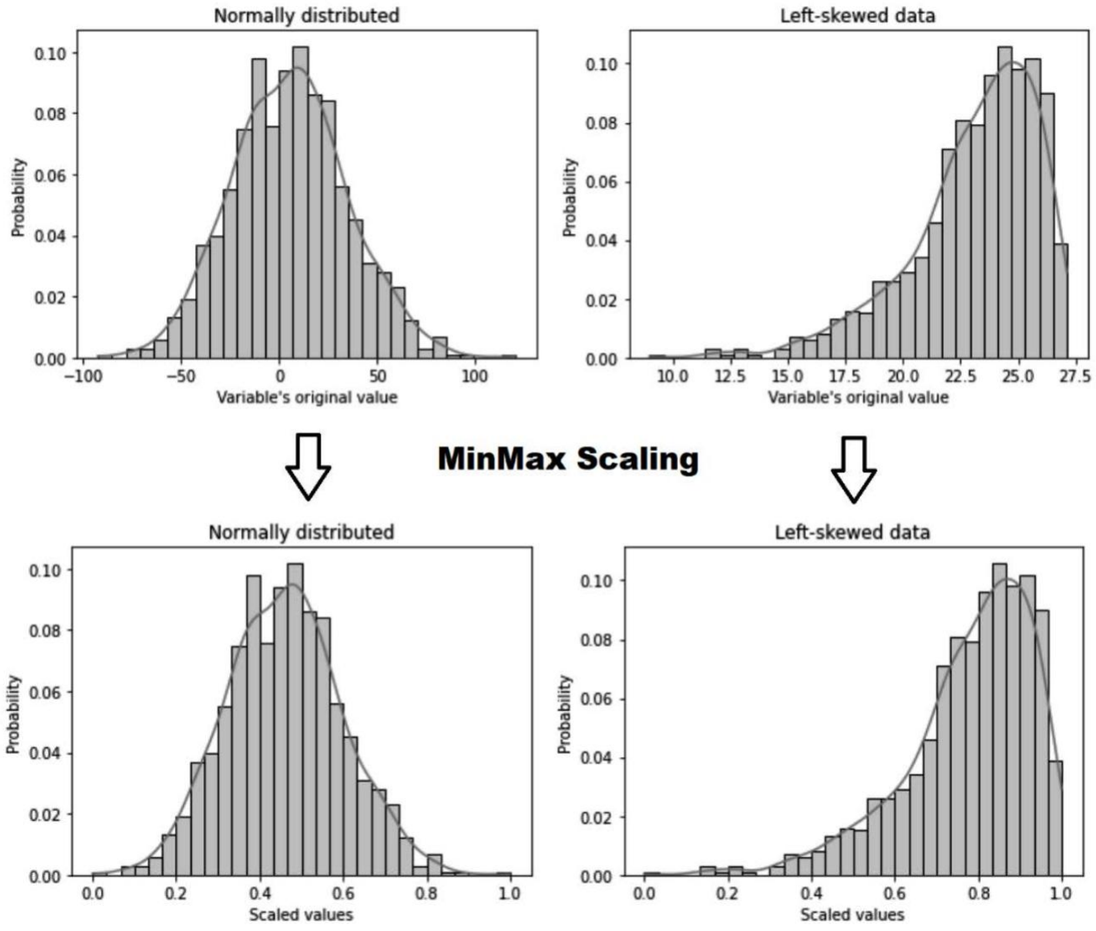
Normalization Methods: Min-Max Scaling

Scales values to range [0,1] or [-1,1]

$$x_scaled = \frac{x - \min(x)}{\max(x) - \min(x)}$$



- Simple, Intuitive
- Preserves distribution shape
- Sensitive to outliers
- May not fit for data with no clear boudaries



Normalization Methods: Min-Max Scaling

Example: Scale the variables of the California housing dataset to values between 0 and 1

```
import pandas as pd
from sklearn.datasets import fetch_california_housing
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import MinMaxScaler
X, y = fetch_california_housing(
    return_X_y=True, as_frame=True)
X.drop(labels=["Latitude", "Longitude"], axis=1,
    inplace=True)
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.3, random_state=0)
scaler = MinMaxScaler().set_output(
    transform="pandas"")
scaler.fit(X_train)
X_train_scaled = scaler.transform(X_train)
X_test_scaled = scaler.transform(X_test)
```

`MinMaxScaler()` stores the maximum and minimum values and the value ranges in its `data_max_`, `min_`, and `data_range_` attributes, respectively

We can corroborate the minimum values of the transformed variables by executing `X_test_scaled.min()`, which will return the following output:

```
MedInc            0.000000
HouseAge          0.000000
AveRooms          0.004705
AveBedrms         0.004941
Population        0.000140
AveOccup         -0.000096
dtype: float64
```

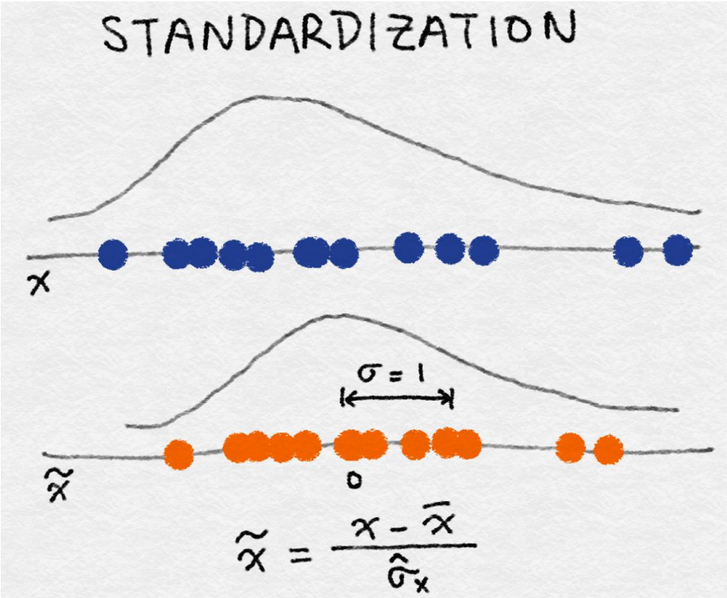
By executing `X_test_scaled.max()`, we see that the maximum values of the variables are around 1:

```
MedInc            1.000000
HouseAge          1.000000
AveRooms          1.071197
AveBedrms         0.750090
Population        0.456907
AveOccup          2.074553
dtype: float64
```

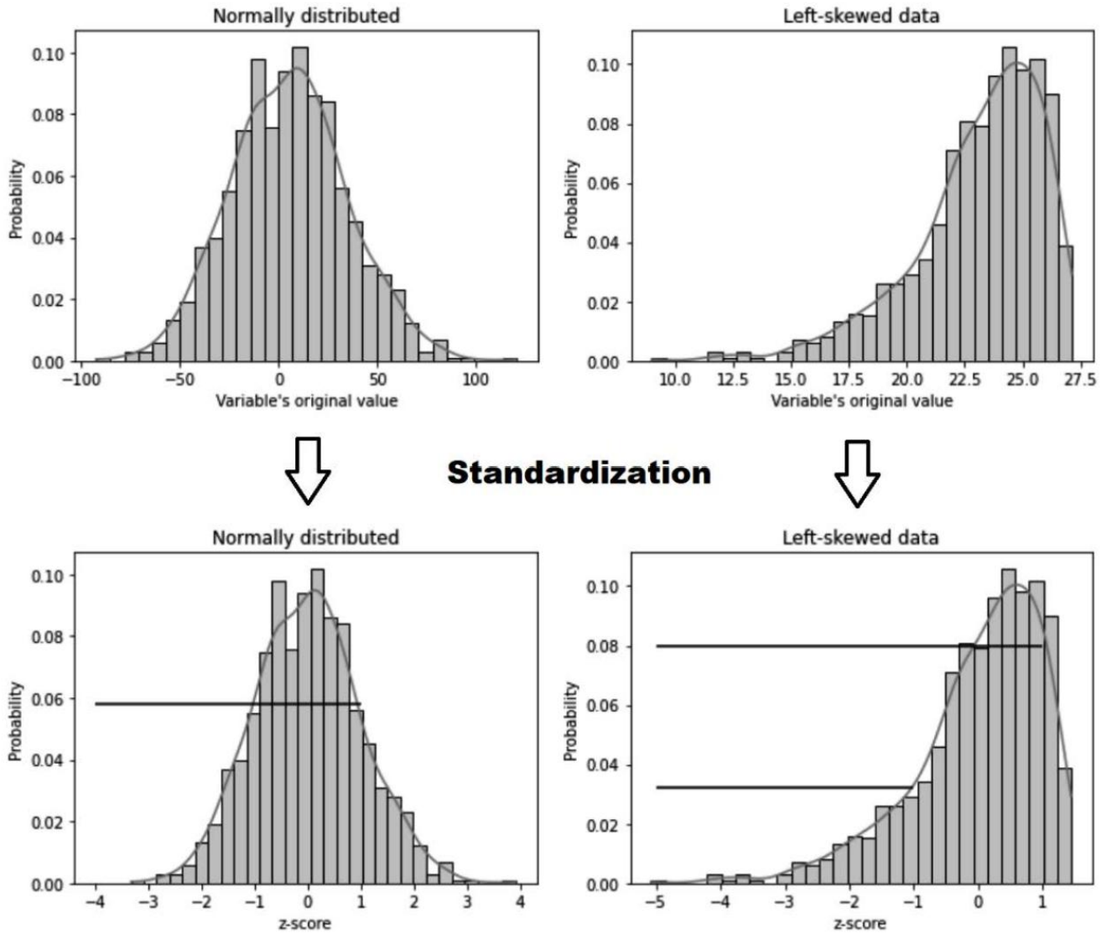
Normalization Methods: Standardization Scaling

Centers the variable at 0 and standardizes the variance to 1

$$x_scaled = \frac{x - \text{mean}(x)}{std(x)}$$



- Less affected by outliers than min-max
- Useful for normally distributed data
- Does not produce bounded range
- Not ideal for non-normal distrubtions



Normalization Methods: Standardization Scaling

Example: Apply standardization to the variables of the California housing dataset

```
import pandas as pd
from sklearn.datasets import fetch_california_housing
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler
X, y = fetch_california_housing(
    return_X_y=True, as_frame=True)
X.drop(labels=["Latitude", "Longitude"], axis=1,
    inplace=True)
```

```
scaler = StandardScaler().set_output(
    transform="pandas")
scaler.fit(X_train)
```

1. Print the **mean values** that were learned by scaler:

```
scaler.mean_
```

⇒ Mean values of each variable in the following output:

```
array([3.86666741e+00, 2.86187016e+01,
5.42340368e+00,                 1.09477484e+00,1.42515732e+03,
3.04051776e+00])
```

2. Print the **standard deviation** values that were learned by scaler:

```
scaler.scale_
```

⇒ Standard deviation of each variable in the following output:

```
array([1.89109236e+00, 1.25962585e+01,
2.28754018e+00,                         4.52736275e-01,
1.14954037e+03, 6.86792905e+00])
```

Normalization Methods: Standardization Scaling

Original variables in the test set =
⇒ Variables' mean values are different from zero and the variance varies:

	MedInc	HouseAge	AveRooms	AveBedrms	Population	AveOccup
count	6192.000000	6192.000000	6192.000000	6192.000000	6192.000000	6192.000000
mean	3.880013	28.687984	5.442057	1.101109	1426.222061	3.140976
std	1.920007	12.560416	2.862733	0.519956	1091.567168	15.796292
min	0.499900	1.000000	1.465753	0.500000	8.000000	0.692308
25%	2.552150	18.000000	4.414452	1.006494	796.000000	2.436452
50%	3.529600	29.000000	5.227365	1.048741	1169.500000	2.825041
75%	4.768750	37.000000	6.064257	1.098434	1727.250000	3.285501
max	15.000100	52.000000	141.909091	25.636364	16305.000000	1243.333333

Values from the transformed variables
⇒ Variables' mean is now centered at 0 and the variance is approximately 1:

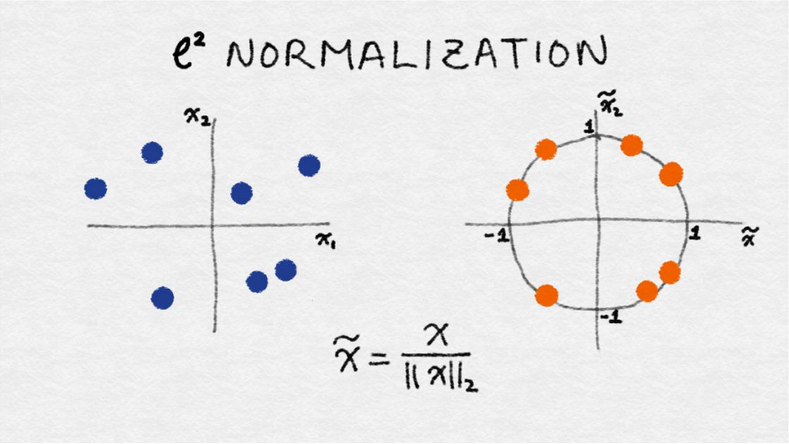
	MedInc	HouseAge	AveRooms	AveBedrms	Population	AveOccup
count	6192.000000	6192.000000	6192.000000	6192.000000	6192.000000	6192.000000
mean	0.007057	0.005500	0.008154	0.013991	0.000926	0.014627
std	1.015290	0.997154	1.251446	1.148474	0.949568	2.300008
min	-1.780329	-2.192612	-1.730090	-1.313734	-1.232803	-0.341909
25%	-0.695110	-0.843004	-0.441064	-0.194995	-0.547312	-0.087955
50%	-0.178240	0.030271	-0.085698	-0.101679	-0.222400	-0.031374
75%	0.477017	0.665380	0.280150	0.008082	0.262794	0.035671
max	5.887302	1.856210	59.664826	54.207251	12.944167	180.591967

Normalization Methods: l_2 Normalization

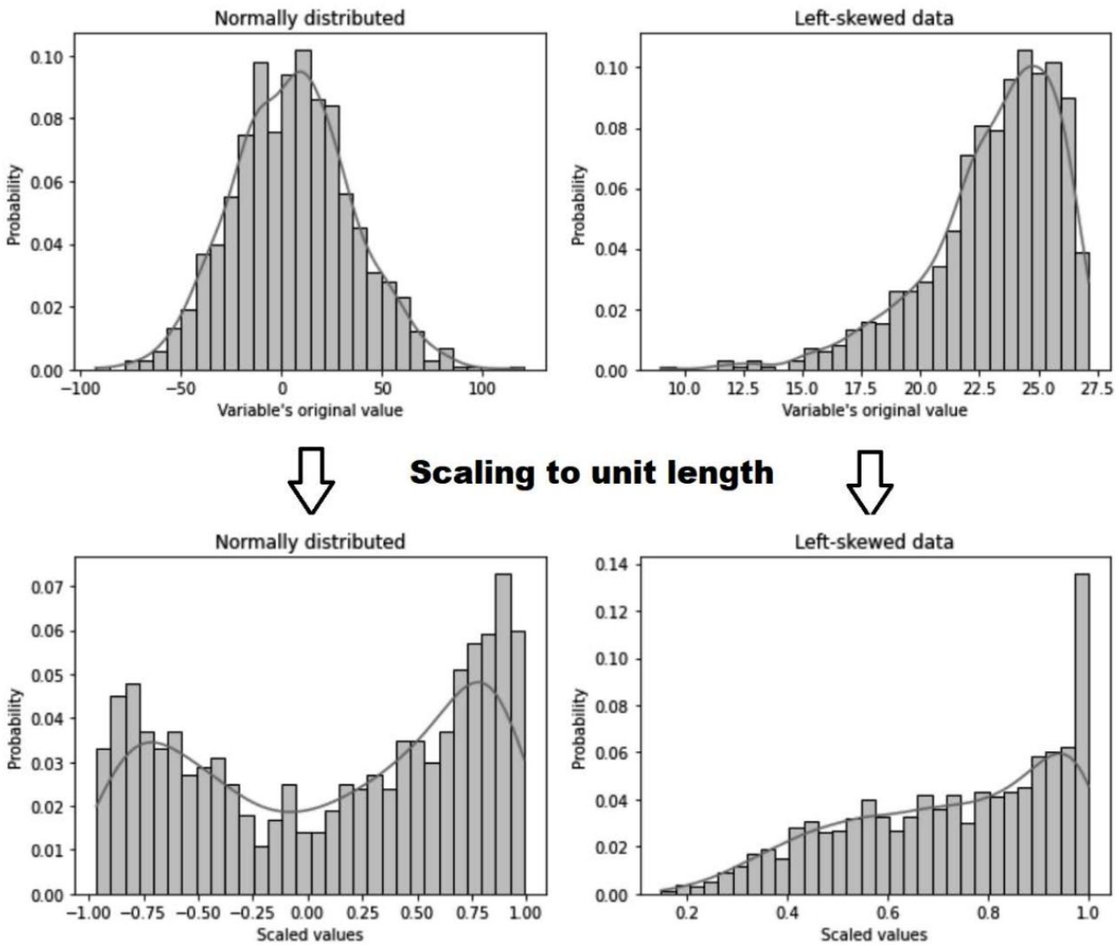
Scales a vector so that its **Euclidean norm** (length) becomes 1, keeping its direction unchanged

$$\tilde{x} = \frac{x}{||x]||_2}$$

$$||x||_2 = \sqrt{x_1{}^2 + x_2{}^2 + \ldots + xm^2}$$



- Standardizes feature scale
- Prevents dominance of features with larger values
- Sensitive to outliers, which can skew normalization



Normalization Methods: l_2 Normalization Practice

Apply l_2 Normalization to the variables of the California housing dataset

https://www.kaggle.com/datasets/camnugent/california-housing-prices

Normalization Methods: Others

Max-absolute scaling

Scales data to the [-1,1] by dividing by max absolute value

$$x_scaled = \frac{x}{max(|x|)}$$

- Handles both positive and negative values
- Keep relative distances between values

- Data is not centered around 0
- Not suitable if features need to be on the same scale

Robust scaling (IQR method)

Uses median & IQR (measure of statistical dispersion)

$$x_scaled = \frac{x - median}{IQR}$$

- Robust to outliers
- Suitable for data with extreme values

- May lose information in normal distributions
- Not as widely implemented in libraries

Data Transformation Techniques: Categorical Encoding

- **Categorical Encoding: The process of converting categorical or text data into numerical format that can be easily understood and processed by algorithms**
- Most machine learning models **require numeric input**

Nominal Categorical Data:

"Represents categories with no inherent order or ranking"

- Categories are distinct and mutually exclusive
- Used to classify items into distinct groups based on shared characteristics
- No mathematical operations can be performed (only equality comparisons)

Examples: colors, gender, animal types, city (London, Manchester, Bristol)

Ordinal Categorical Data: *"Represents categories with a meaningful order or ranking"*

- Categories have relative positions, but intervals aren't necessarily uniform
- Allows for comparison operations (greater/less than)
- Cannot precisely quantify differences between categories

Examples: education levels, satisfaction ratings (poor/good/excellent)

- Multiple encoding methods exist and the choice of encoding technique depends on the type of categorical data

Categorical Encoding Methods : One-Hot Encoding

Converts each categorical value into a binary column (0/1)

Color
Red
Green
Blue

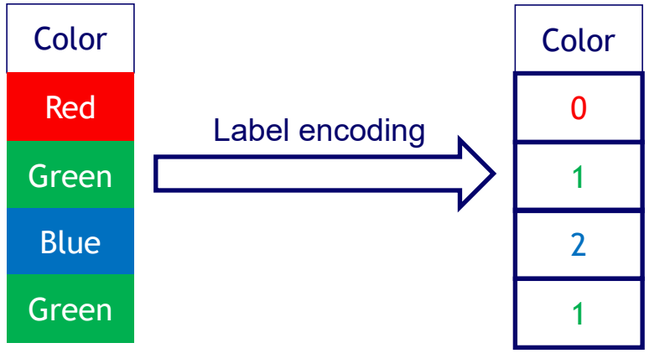
One-hot encoding →

d1	d2	d3
1	0	0
0	1	0
0	0	1

- Nominal data with low cardinality, used in linear models and neural networks
- Can lead to high dimensionality if many unique categories

Categorical Encoding Methods : Label Encoding

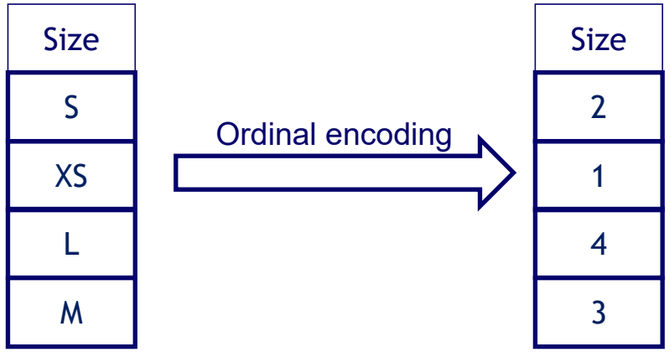
Assigns a unique integer to each category



- Best for Ordinal data or tree-based models (e.g., Decision Trees, Random Forests)
- Caution: Can mislead linear models into thinking that order has meaning

Categorical Encoding Methods : Ordinal Encoding

Similar to label encoding but used only when the categories have a clear order



- Captures ordinal relationships
- Wrong usage can mislead models if order doesn't exist

Categorical Encoding Methods : Target Encoding

Replaces categories with the mean of the target variable for that category

workclass	target
State-gov	0
Self-emp-not-inc	1
Private	0
Private	0
Private	1

workclass	target mean
State-gov	0
Self-emp-not-inc	1
Private	1/3

workclass
0
1
1/3
1/3
1/3

- Directly captures relationship between categories and target
- Handles high-cardinality features extremely well
- Creates meaningful numerical values
- Works with any ML algorithm

- Can cause data leakage if not done carefully
- Sensitive to outliers in the target

Categorical Encoding Methods : Frequency Encoding

Replaces each category with its frequency or count in the dataset

Color
Red
Green
Blue
Green

Count encoding →

Count	Freq
1	0.25
2	0.5
1	0.25
2	0.5

- Keep information compact
- Preserves dimensionality (one column stays one column)
- May give too much weight to rare or common categories
- Less interpretable than original categories

Why we need Numerical Variable Transformations

- In most, general linear model, it is assumed that there is a linear relationship between the target and the independent (predictor) variables.

- When, the data is skewed, this assumption is not met, the resulting probabilistic statements might not be accurate.

- Transforming variables with mathematical functions helps reduce variable skewness, improves the value spread, and sometimes unmasks linear and additive relationships between predictors and the target.

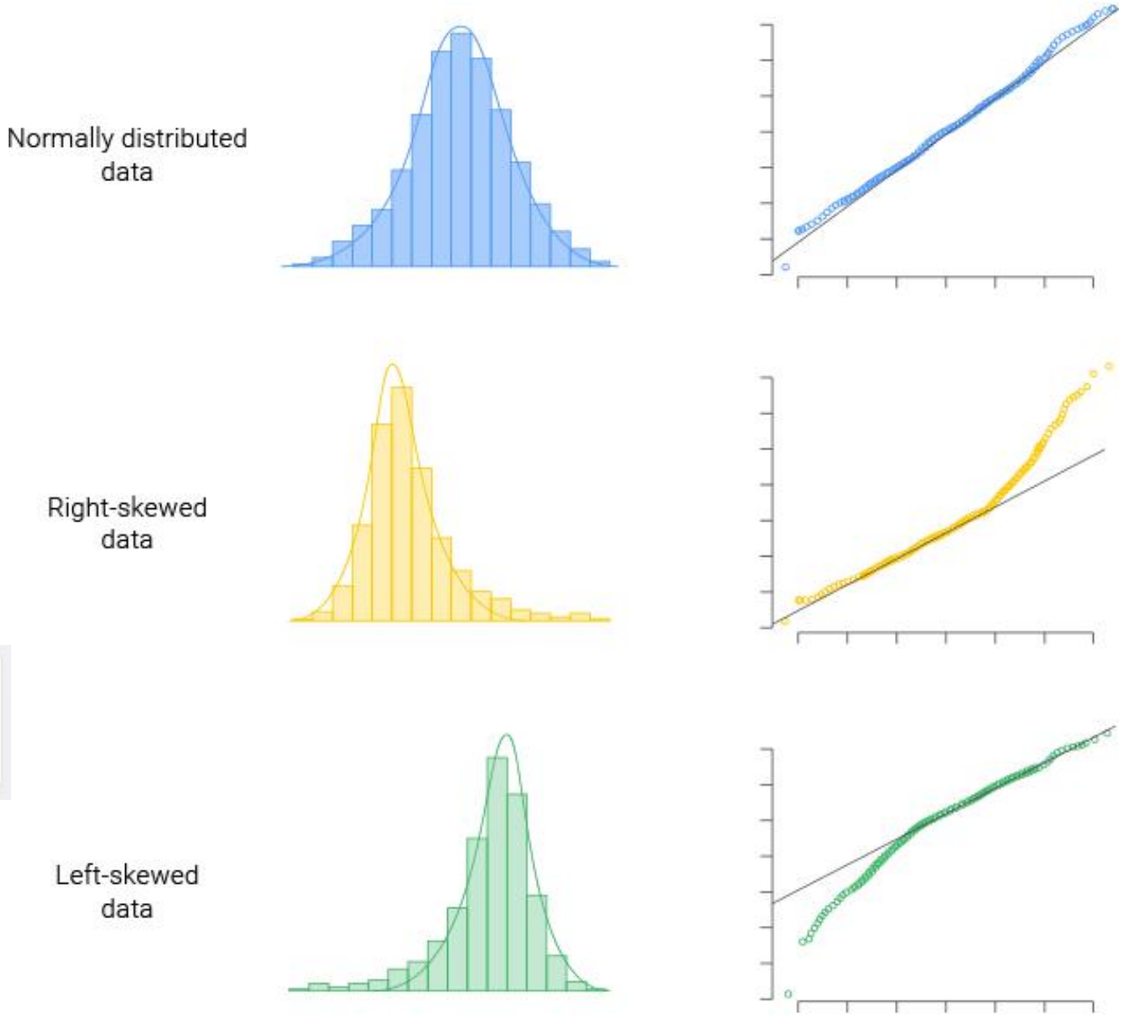
- **Transformation**:
  - Logarithm function
  - Reciprocal function
  - The square root
  - Power
  - Box-Cox transformations
  - Yeo-Johnson transformations

Q-Q plot

- A Q-Q plot (Quantile-Quantile plot) is a graphical tool used in statistics to assess if a dataset follows a particular theoretical distribution, such as the normal distribution.
- It does so by plotting the quantiles of the observed data against the quantiles of the theoretical distribution.

```python
from sklearn.datasets import fetch_california_housing
X, y = fetch_california_housing(return_X_y=True, as_frame=True)
```

	MedInc	HouseAge	AveRooms	AveBedrms	Population	AveOccup	Latitude	Longitude
0	2.232720	41.0	6.984127	1.023810	322.0	2.555556	37.88	-122.23
1	2.230165	21.0	6.238137	0.971880	2401.0	2.109842	37.86	-122.22
2	2.111110	52.0	8.288136	1.073446	496.0	2.802260	37.85	-122.24
3	1.893579	52.0	5.817352	1.073059	558.0	2.547945	37.85	-122.25

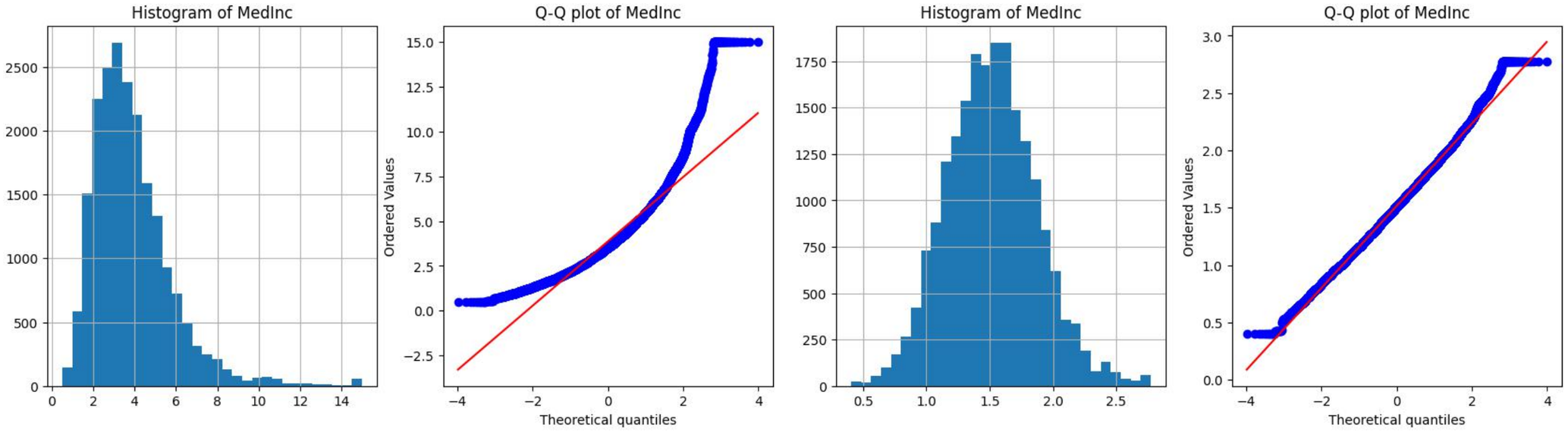


Logarithmic Transformation

- Applies the natural logarithm **log(x+1)** to each data value.
- When data exhibit **right skewness** or have a **long right tail**.
- Reduces right skewness, making the data more symmetric.

```python
def diagnostic_plots(df, variable):
  plt.figure(figsize=(10,5))
  plt.subplot(1, 2, 1)
  df[variable].hist(bins=30)
  plt.title(f"Histogram of {variable}")
  plt.subplot(1, 2, 2)
  stats.probplot(df[variable], dist="norm", plot=plt)
  plt.title(f"Q-Q plot of {variable}")
  plt.show()
```

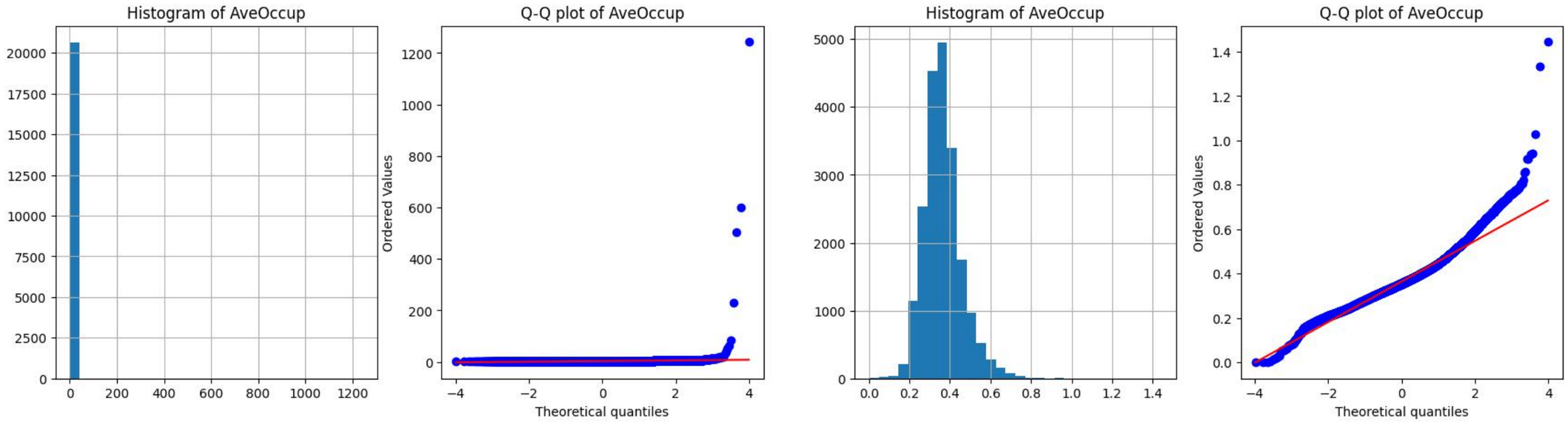
```python
X["MedInc"] = np.log(X["MedInc"]+1)
diagnostic_plots(X, "MedInc")
```



Reciprocal function

- The reciprocal function is defined as **1/x**
- When data shows very strong right-skewed distribution.

```
X["AveOccup"] = np.reciprocal(X["AveOccup"])
```



- **Square Root:** $\sqrt{x}$, when dealing with count data.
- **Power :** x^{λ} when others are not sufficiently normalizing the data.

Box-Cox Transformation

- A generalization of the power transformations, finds the optimal lambda value via maximum likelihood.

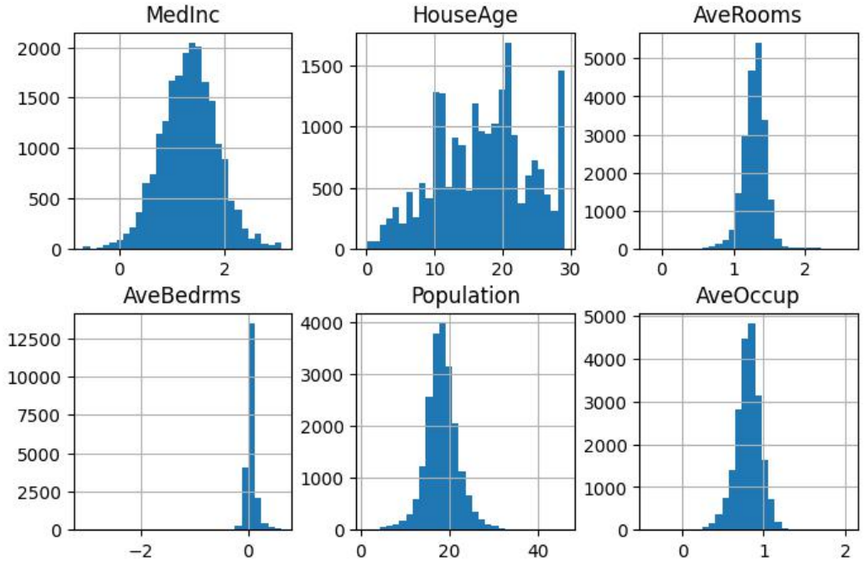
$$y^{(\lambda)} = \frac{(y^{\lambda} - 1)}{\lambda} \; if \; \lambda \neq 0$$

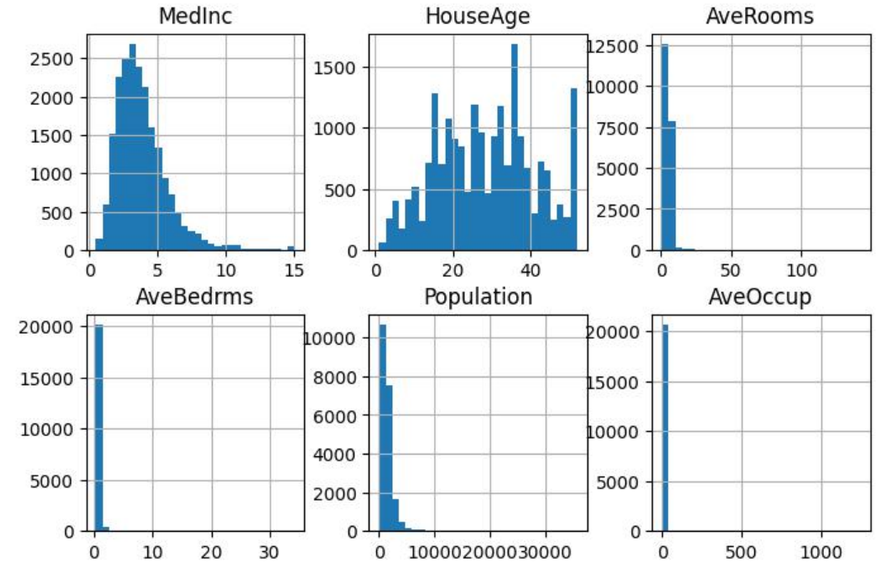
$$y^{(\lambda)} = \log(y) \; if \; \lambda = 0$$

- The Box-Cox transformation evaluates several values of λ using the maximum likelihood and selects the λ parameter that returns the best transformation.

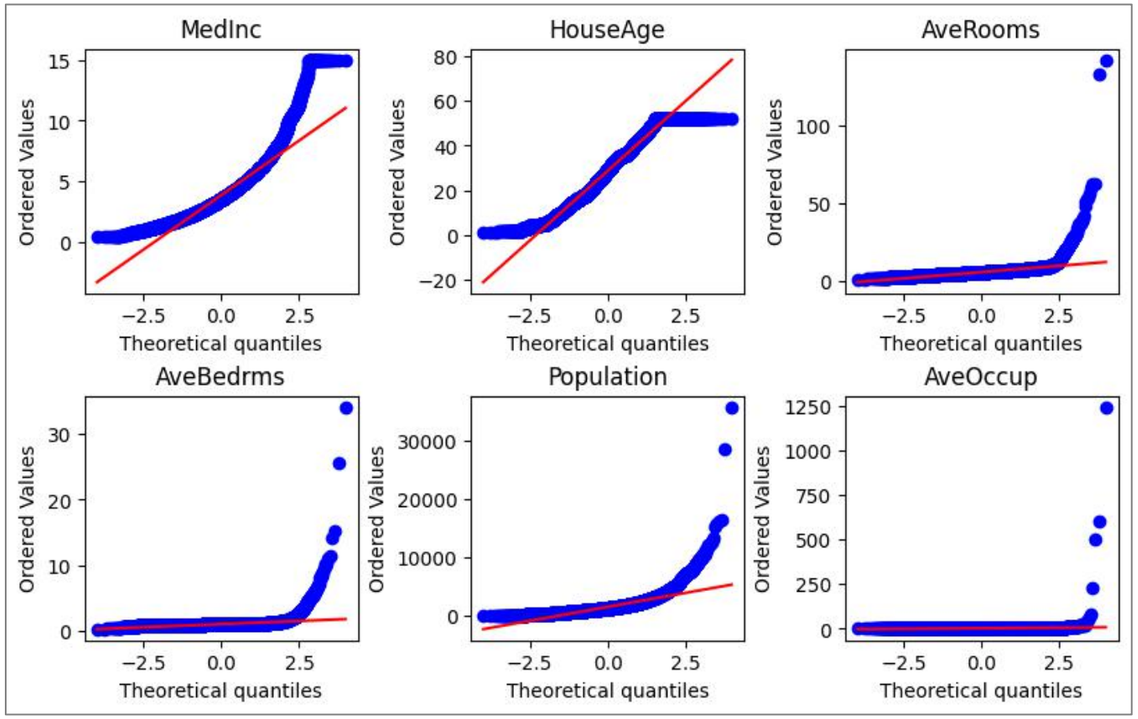
```
!pip install feature-engine
```

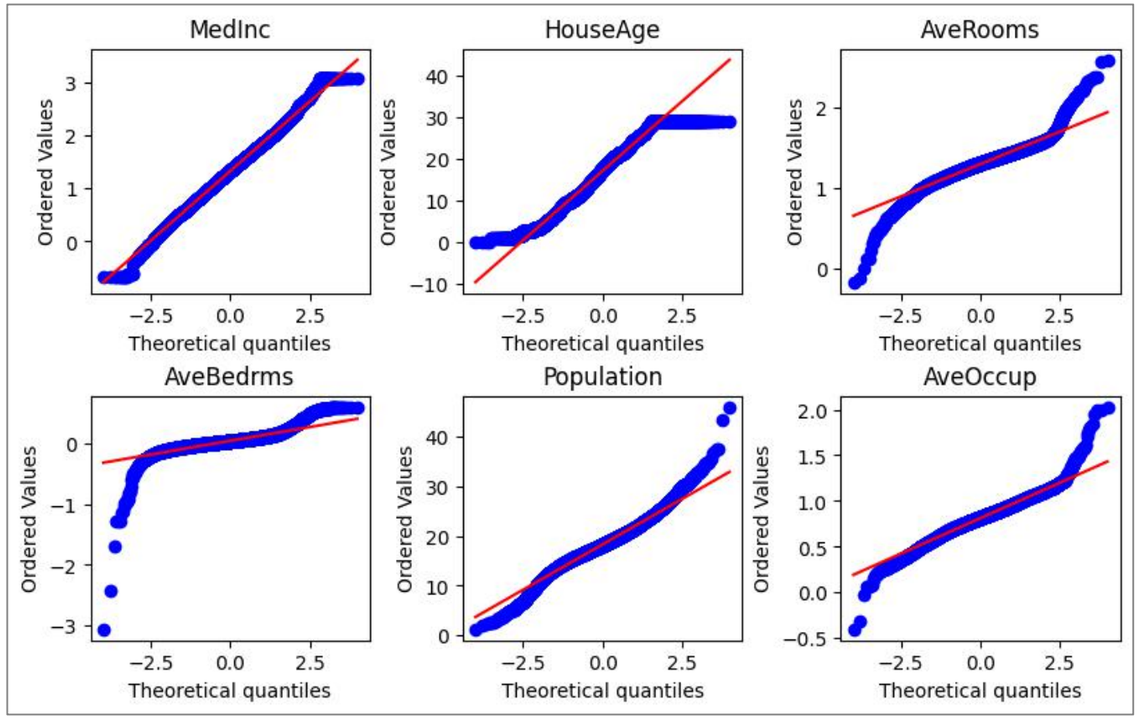
```
from feature_engine.transformation import BoxCoxTransformer
bct = BoxCoxTransformer()
bct.fit(X)
X_tf = bct.transform(X)
bct.lambda_dict_
# output
{'MedInc': 0.090854472181,
 'HouseAge': 0.8093980693938718,
 'AveRooms': -0.29800490205071034,
 'AveBedrms': -1.6290002880977101,
 'Population': 0.2357675549594364,
 'AveOccup': -0.4763032672955569}
```





Box-Cox Transformation





One issue with Box-Cox, is not suitable for data containing zero or negative values

Yeo-Johnson transformations

- The Yeo-Johnson transformation is an extension of the Box-Cox transformation.
- Can be used on variables with zero and negative values, as well as positive values.

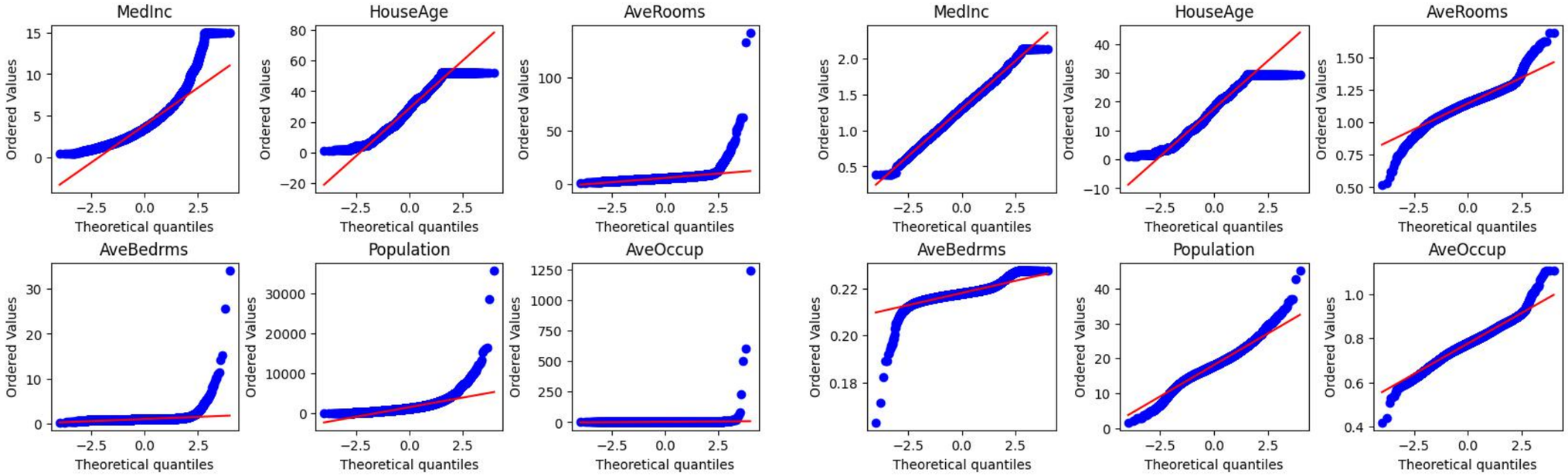
$\frac{(X+1)^{\lambda}-1}{\lambda}$; if $\lambda \neq 0$ and $X >= 0$

$\ln(X+1)$; if $\lambda = 0$ and $X >= 0$

$-\frac{(-X+1)^{2-\lambda}-1}{2-\lambda}$; if $\lambda \neq 2$ and $X < 0$

$-\ln(-X+1)$; if $\lambda = 2$ and $X < 0$

```python
from feature_engine.transformation import YeoJohnsonTransformer
yjt = YeoJohnsonTransformer()
yjt.fit(X)
X_tf = yjt.transform(X)
```



Practical Labs Module 4